

1-Task Title: Different Types of Prime Check:

Code:

```
import math
integer=[]
def is_prime(n):
    if n in integer:
        return True
    return False

def mersene(n):
    x=int(math.log2(n+1))
    return (is_prime(x) and 2**x==n+1)

def twin(n):
    return is_prime(n)

with open("input.txt","r") as file:
    a=file.readlines()
    for i in a:
        integer.append(int(i))
    file.close()

a=int(input("Write an Integer-> "))

if is_prime(a):
    print(f"{a} is a prime number\n")
    if mersene(a):
        print(f"{a} is a mersene prime number\n")
    else:
        print(f"{a} is not a mersene prime number\n")
    if(twin(a+2)):
        print(f"{a} has twin prime\n")
    else:
        print(f"{a} has not twin prime\n")
else:
    print(f"{a} is not a prime number\n")
```

Output:

```
Write an Integer-> 31
31 is a prime number

31 is a mersene prime number

31 has not twin prime

PS F:\RUET CSE LAB\Discrete Mathematics 2102\7. 24-02-2024>
```

```

o Write an Integer-> 5
  5 is a prime number

  5 is not a mersene prime number

  5 has twin prime

PS F:\RUET CSE LAB\Discrete Mathematics 2102\7. 24-02-2024>

```

Discussion:

In this program, I have used a file to store the prime numbers. For storing prime numbers, first I created a file and used the method of Sieve of Eratosthenes to remove the composite numbers. Then took the input from file and did the calculation as per rules for checking just primes, Mersene Primes and Twin Primes.

2-Task Title: GCD Calculation using Prime Factorization:

Code:

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define newl cout<<endl

map<int,int>Prime_divisor(int a)
{
    map<int,int>mp;
    int r=2;
    while(a!=1)
    {
        if(a%r==0)
        {
            mp[r]++;
            a/=r;
        }
        else
        {
            r+=1;
        }
    }
    return mp;
}

void print(map<int,int>mp)
{
    for(auto i:mp)
    {
        cout<<i.first<<"^"<<i.second<<"*";
    }
    cout<<'1';
    newl;newl;
}

```

```

int main()
{
    cout<<"Write 2 integers-> ";
    int a,b;
    cin>>a>>b;
    map<int,int>mpa=Prime_divisor(a),mpb=Prime_divisor(b);
    newl;
    cout<<a<<" = ";print(mpa);
    cout<<b<<" = ";print(mpb);
    int gcd=1;
    for(int i=2;i<=min(sqrt(a),sqrt(b));i++)
    {
        gcd*=pow(i,min(mpa[i],mpb[i]));
    }
    cout<<"the GCD result is-> "<<gcd;newl;
}

```

Output:

```

Write 2 integers-> 12 28

12 = 2^2*3^1*1

28 = 2^2*7^1*1

the GCD result is-> 4
PS F:\RUET CSE LAB\Discrete Mathematics 2102\7. 24-02-2024>

```

Discussion:

Here I have used prime factorization techniques to calculate the Greatest Common Divisor. In this method, I must express a composite number in the term of prime numbers. As a result, when calculating the gcd I must just take the minimum numbers of a prime factor from the expression and then multiply them accordingly to get the gcd.

3-Task Title: GCD Calculation using Euclidean Algorithm:

Code:

```

def _gcd(a,b):
    if b==0:
        return a;
    return _gcd(b,a%b)

a,b=map(int,input("Write 2 integers-> ").split(" "))
print(f"\nThe GCD of {a} and {b} is {_gcd(a,b)}")

```

Output:

```
Write 2 integers-> 12 28
```

```
The GCD of 12 and 28 is 4
```

```
PS F:\RUET CSE LAB\Discrete Mathematics 2102\7. 24-02-2024>
```

Discussion:

It is also a gcd calculating technique where I must just calculate the modulus value of each number. When performing this action, I have used recursive function where the base case is 'if $b==0$ ', else, first value is replaced by second value and second value is replaced by mod value of them and it will continue until it hits the base case. In this method we can get the gcd value of 2 numbers easily.